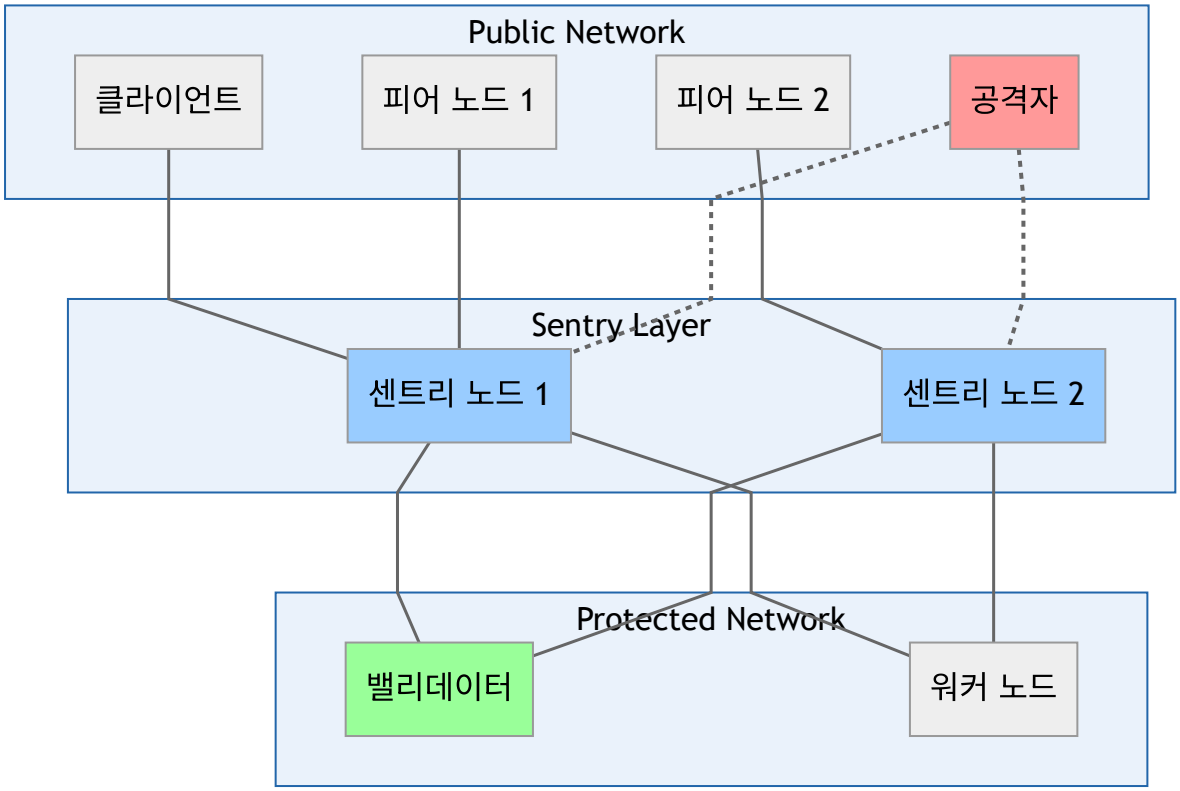
센트리 노드 (Sentry Node)

소개

센트리 노드는 Oasis Core 네트워크에서 밸리데이터와 다른 중요한 노드들을 보호하는 특수한 네트워크 게이트웨이 역할을 합니다. 센트리 노드는 밸리데이터의 실제 네트워크 주소를 숨기고, 네트워크 공격으로부터 보호하며, 안전한 통신 채널을 제공합니다.

보안 아키텍처

네트워크 격리 모델



보안 계층

1. 네트워크 계층 보호

- **IP 주소 은닉**: 밸리데이터의 실제 IP 주소 숨김
- **DDoS 방어**: 분산 서비스 거부 공격 완화
- **트래픽 필터링**: 악의적인 트래픽 차단

2. 인증 계층

- **상호 TLS 인증**: 클라이언트-센트리 간 인증서 기반 인증
- **공개키 화이트리스트**: 승인된 업스트림 노드만 접근 허용
- **세션 관리**: 안전한 연결 세션 유지

3. 프로토콜 계층

- **메시지 검증**: 프로토콜 준수 검증
- **속도 제한**: 요청 빈도 제한
- **로깅 및 모니터링**: 모든 접근 기록

센트리 노드 구성 요소

1. Sentry Backend

```go
// 센트리 백엔드 인터페이스
type Backend interface {
// GetAddresses는 센트리 노드의 합의 및 TLS 주소 목록을 반환합니다
GetAddresses(context.Context) (*SentryAddresses, error)
}

// 센트리 주소 구조체
```

```go
type SentryAddresses struct {
Consensus []node.ConsensusAddress `json:"consensus"`
}
```

기능:

- 업스트림 노드 주소 관리
- 주소 정보 제공 서비스
- 동적 주소 업데이트

2. Sentry Worker

```go
// 센트리 워커 구조체
type Worker struct {
enabled  bool
sentry api.Backend
grpcServer *grpc.Server
logger *logging.Logger
}

// 센트리 워커 시작
func (w *Worker) Start() error {
if !w.enabled {
w.logger.Info("센트리 워커가 비활성화되어 시작하지 않습니다")
return  nil
```

```go
}

// 센트리 gRPC 서버 시작

if err := w.grpcServer.Start(); err != nil {

return fmt.Errorf("센트리 gRPC 서버 시작 실패: %w", err)

}

return nil

}
```

3. Control Endpoint

```go
// 센트리 제어 설정

type ControlConfig struct {

// 센트리 워커의 gRPC 서버 포트

Port uint16 `yaml:"port"`

// 센트리 제어 엔드포인트에 연결할 수 있는 업스트림 노드의 공개키

AuthorizedPubkeys []string `yaml:"authorized_pubkeys"`

}
```

특징:

- 승인된 노드만 접근 가능
- 주소 정보 쿼리 서비스
- 실시간 상태 모니터링

밸리데이터 보호 메커니즘

1. 주소 은닉

```
// 밸리데이터 주소 수집 (센트리를 통해)
func (w *Worker) gatherConsensusAddresses(sentryConsensusAddrs []node.ConsensusA
var consensusAddrs []node.ConsensusAddress
switch len(w.sentryAddresses) > 0 {
case true:
// 센트리 노드가 사용되는 경우, 센트리 주소 사용
consensusAddrs = sentryConsensusAddrs
case false:
// 그렇지 않으면 직접 합의 주소 수집
// ... 직접 주소 수집 로직
}
return consensusAddrs, nil
}
```

2. 연결 프록시

```
// 센트리를 통한 연결 설정
func (w *Worker) querySentries() []node.ConsensusAddress {
var sentryAddresses []node.ConsensusAddress
for _, sentryAddr := range w.sentryAddresses {
client, err := sentryClient.New(sentryAddr, w.identity)
```

```go
if  err != nil {
w.logger.Warn("센트리 노드 클라이언트 생성 실패",
"err", err,
"sentry_address", sentryAddr,
)
continue
}
defer  client.Close()
// 센트리 노드에서 주소 쿼리
addresses, err := client.GetAddresses(w.ctx)
if  err != nil {
w.logger.Warn("센트리 노드에서 주소 가져오기 실패",
"err", err,
"sentry_address", sentryAddr,
)
continue
}
sentryAddresses = append(sentryAddresses, addresses.Consensus...)
}
return  sentryAddresses
}
```

3. 트래픽 필터링

```go
// 센트리 클라이언트 인증
func (c *Client) createConnection() error {
// 보안 gRPC 연결 설정
creds, err := cmnGrpc.NewClientCreds(&cmnGrpc.ClientOptions{
CommonName: identity.CommonName,
ServerPubKeys: map[signature.PublicKey]bool{
c.sentryAddress.PubKey: true,
},
Certificates: []tls.Certificate{*c.identity.TLSSentryClientCertificate},
})
if err != nil {
return err
}
conn, err := cmnGrpc.Dial(c.sentryAddress.String(), grpc.WithTransportCredential
if err != nil {
return fmt.Errorf("센트리 노드 연결 실패: %w", err)
}
c.conn = conn
c.Backend = api.NewClient(conn)
return nil
}
```

센트리 노드 설정

기본 설정

```yaml
# 센트리 노드 설정
sentry:
# 센트리 워커 활성화 (센트리 노드에서만 활성화해야 함)
enabled: true
# 제어 엔드포인트 설정
control:
# 센트리 워커의 gRPC 서버 포트
port: 9001
# 센트리 제어 엔드포인트에 연결할 수 있는 업스트림 노드의 공개키
authorized_pubkeys:
- "oasis1qp0l8r2s3076n4xasl72qx0dc5s4c7d2rqjcqrjd"
- "oasis1qp5l8r2s3076n4xasl72qx0dc5s4c7d2rqjcqrje"

# 합의 설정 (센트리 노드용)
consensus:
# 센트리 노드가 연결할 업스트림 주소
sentry_upstream_addresses:
- "oasis1qp0l8r2s3076n4xasl72qx0dc5s4c7d2rqjcqrjd@192.168.1.100:26656"
```

밸리데이터 설정

```yaml
# 밸리데이터 노드 설정 (센트리 사용)
runtime:
# 센트리 노드 주소 목록
sentry_addresses:
- "cert_hash@192.168.1.200:9001"
- "cert_hash@192.168.1.201:9001"


# 합의 설정 (밸리데이터용)
consensus:
# 외부 주소를 설정하지 않음 (센트리를 통해 접근)
# external_address: ""
# 센트리 노드만 연결 허용
listen_address: "127.0.0.1:26656"
```

고급 보안 설정

```yaml
# 고급 센트리 설정
sentry:
enabled: true
control:
port: 9001
authorized_pubkeys:
- "validator1_pubkey"
```

```yaml
- "validator2_pubkey"

# TLS 설정

tls:

# 클라이언트 인증서 요구

client_auth: require

# 허용된 클라이언트 인증서 CA

client_ca_file: "/path/to/client-ca.pem"

# 서버 인증서

cert_file: "/path/to/sentry-cert.pem"

key_file: "/path/to/sentry-key.pem"



# 네트워크 보안 설정

p2p:

# 연결 제한

connection_manager:

max_num_peers: 50

# 지속적 피어 (업스트림 노드)

persistent_peers:

- "validator1@127.0.0.1:26656"

- "validator2@127.0.0.1:26656"

# 차단된 피어

blocked_peers:
```

```
- "malicious_peer_id"
```

센트리 노드 운영

1. 센트리 노드 시작

```
# 센트리 노드 시작
oasis-node  --config  /path/to/sentry-config.yml

# 로그에서 센트리 서비스 확인
# "센트리 gRPC 서버 시작됨" 메시지 확인
```

2. 상태 확인

```
# 센트리 노드 상태 확인
oasis-node  control  status

# 센트리 제어 엔드포인트 테스트
grpcurl  -plaintext  localhost:9001  oasis-core.Sentry/GetAddresses
```

3. 연결 모니터링

```
// 센트리 연결 상태 모니터링
func  monitorSentryConnections(sentryAddresses []node.TLSAddress) {
```

```
for _, addr := range sentryAddresses {
client, err := sentryClient.New(addr, identity)
if err != nil {
log.Printf("센트리 %s 연결 실패: %v", addr, err)
continue
}
addresses, err := client.GetAddresses(context.Background())
if err != nil {
log.Printf("센트리 %s 주소 조회 실패: %v", addr, err)
continue
}
log.Printf("센트리 %s 연결 성공, 주소 수: %d", addr, len(addresses.Consensus))
client.Close()
}
}
```

API 및 인터페이스

Sentry API

```
// 센트리 서비스 인터페이스
service Sentry {
// 센트리 노드의 주소 목록 조회
```

```
rpc GetAddresses(google.protobuf.Empty) returns (SentryAddresses);
}

// 센트리 주소 응답
message SentryAddresses {
repeated ConsensusAddress consensus = 1;
}

// 합의 주소 구조체
message ConsensusAddress {
string id = 1; // 노드 ID
string address = 2; // 네트워크 주소
}
```

클라이언트 사용 예제

```
// 센트리 클라이언트 생성 및 사용
func useSentryClient() error {
// 센트리 주소 설정
sentryAddr := node.TLSAddress{
PubKey: sentryPubKey,
Address: "192.168.1.200:9001",
}
```

```go
// 클라이언트 생성
client, err := sentryClient.New(sentryAddr, nodeIdentity)
if err != nil {
return fmt.Errorf("센트리 클라이언트 생성 실패: %w", err)
}
defer client.Close()
// 주소 조회
addresses, err := client.GetAddresses(context.Background())
if err != nil {
return fmt.Errorf("주소 조회 실패: %w", err)
}
// 결과 처리
for _, addr := range addresses.Consensus {
fmt.Printf("합의 주소: %s\n", addr)
}
return nil
}
```

모니터링 및 로깅

로그 설정

```
# 센트리 관련 로깅 설정
```

```yaml
log:
level: info
format: JSON
# 센트리 특정 로깅
modules:
sentry: debug
sentry/client: debug
sentry/worker: info
```

메트릭 수집

```go
// 센트리 메트릭 정의
var (
sentryConnections = prometheus.NewGaugeVec(
prometheus.GaugeOpts{
Name: "oasis_sentry_connections_total",
Help: "센트리 노드 연결 수",
},
[]string{"sentry_address"},
)
sentryRequests = prometheus.NewCounterVec(
prometheus.CounterOpts{
Name: "oasis_sentry_requests_total",
```

```
Help: "센트리 노드 요청 수",
},
[]string{"sentry_address", "method"},
)
)

// 메트릭 업데이트
func updateSentryMetrics(addr string, connected bool) {
if connected {
sentryConnections.WithLabelValues(addr).Set(1)
} else {
sentryConnections.WithLabelValues(addr).Set(0)
}
}
```

알림 설정

```
# Prometheus 알림 규칙
groups:
- name: sentry
rules:
- alert: SentryNodeDown
expr: oasis_sentry_connections_total == 0
```

```yaml
for: 5m
labels:
severity: critical
annotations:
summary: "센트리 노드 연결 끊김"
description: "센트리 노드 {{ $labels.sentry_address }}에 연결할 수 없습니다"
- alert: SentryHighLatency
expr: oasis_sentry_request_duration_seconds > 1
for: 2m
labels:
severity: warning
annotations:
summary: "센트리 노드 응답 지연"
description: "센트리 노드 응답 시간이 1초를 초과합니다"
```

보안 모범 사례

1. 네트워크 분리

- 센트리 노드를 DMZ에 배치
- 밸리데이터를 내부 네트워크에 격리
- 방화벽 규칙으로 접근 제어

2. 인증서 관리

```
# 센트리용 TLS 인증서 생성
oasis-node identity init --datadir /path/to/sentry

# 클라이언트 인증서 생성
oasis-node identity init-client --datadir /path/to/client

# 인증서 교환 및 검증
oasis-node identity show-sentry-client-pubkey --datadir /path/to/client
```

3. 접근 제어

```yaml
# 센트리 접근 제어 설정
sentry:
control:
# IP 기반 접근 제어
allowed_ips:
- "192.168.1.0/24"
- "10.0.0.0/8"
# 공개키 기반 인증
authorized_pubkeys:
- "validator1_pubkey"
- "validator2_pubkey"
# 속도 제한
```

```yaml
rate_limit:
  requests_per_second: 10
  burst_size: 20
```

문제 해결

일반적인 문제

1. 센트리 연결 실패

```bash
# 네트워크 연결 확인
telnet sentry-address 9001
# 인증서 확인
openssl s_client -connect sentry-address:9001 -cert client.crt -key client.key
```

2. 인증 실패

```bash
# 공개키 확인
oasis-node identity show-sentry-client-pubkey --datadir /path/to/client
# 센트리 설정에서 공개키 확인
grep -A 10 "authorized_pubkeys" /path/to/sentry-config.yml
```

3. 성능 문제

```bash
# 센트리 로드 확인
```

```
oasis-node control status | grep connections

# 네트워크 지연 측정

ping sentry-address
```

디버깅 도구

```
# 센트리 상태 확인

oasis-node debug sentry status


# 센트리 연결 테스트

oasis-node debug sentry test-connection --address sentry-address:9001


# 센트리 로그 분석

tail -f /var/log/oasis/sentry.log | grep -E "(ERROR|WARN)"
```

고가용성 설정

다중 센트리 구성

```
# 밸리데이터 설정 (다중 센트리)

runtime:

sentry_addresses:

- "sentry1_cert@192.168.1.200:9001"

- "sentry2_cert@192.168.1.201:9001"
```

```yaml
- "sentry3_cert@192.168.1.202:9001"


# 로드 밸런싱 설정
sentry:
load_balancing:
strategy: "round_robin"  # round_robin, least_connections, random
health_check_interval: "30s"
retry_attempts: 3
```

장애 복구

```go
// 센트리 장애 복구 로직
func (w *Worker) handleSentryFailover() {
for  _, sentryAddr := range  w.sentryAddresses {
client, err := sentryClient.New(sentryAddr, w.identity)
if  err != nil {
w.logger.Warn("센트리 연결 실패, 다음 센트리 시도",
"failed_sentry", sentryAddr,
)
continue
}
// 연결 성공 시 사용
w.activeSentryClient = client
```

```
w.logger.Info("센트리 연결 성공",
"active_sentry", sentryAddr,
)
return
}
w.logger.Error("모든 센트리 노드 연결 실패")
}
```

이 문서는 Oasis Core의 센트리 노드 시스템에 대한 포괄적인 가이드를 제공합니다. 센트리 노드를 통해 밸리데이터와 중요한 노드들을 네트워크 공격으로부터 보호하고, 안전한 네트워크 아키텍처를 구축할 수 있습니다.